

Historia de usuario M6.1S1

Cimiento Arquitectónico de "Eventify" - Catálogo Base y Estereotipos

Objetivo de la Historia de usuario

Implementar la estructura inicial de la plataforma "Eventify" aplicando el patrón Spring MVC, gestionando el ciclo de vida de los Beans mediante Estereotipos e integrando prácticas profesionales de Documentación (Swagger) y Testing Unitario desde el primer día.

Contexto de Negocio

Para que "Eventify" sea una plataforma escalable, necesitamos una base técnica sólida. La primera fase del negocio consiste en poder registrar los lugares (Venues) y los eventos (Events) de forma interna. Aunque los datos serán temporales (en memoria), debemos asegurar que la comunicación entre las capas de la aplicación sea limpia y que el sistema sea confiable antes de añadir persistencia real.

Detalle de la HU

- **Como:** Administrador del sistema
- **Quiero:** Registrar y visualizar tanto eventos, como lugares de forma organizada
- **Para:** Validar la lógica inicial del catálogo y asegurar la estabilidad de la arquitectura base.

TASK 1

Configuración de Ecosistema y Starters

1. **Inicialización:** Crear el proyecto utilizando Spring Initializr.
2. **Dependencias (Starters):** Incluir `spring-boot-starter-web` (para el ecosistema MVC) y `lombok` (para reducir código de plomería).
3. **Configuración de Beans:** Implementar una clase `@Configuration` que utilice la anotación `@Bean` para cargar datos iniciales (Seeders) en el arranque de la aplicación.
4. **Inyección de Dependencias:** Aplicar estrictamente Inyección por Constructor en todos los componentes para facilitar el desacoplamiento y el testeo.

TASK 2

Implementación de Estereotipos y Capas (MVC)

1. **Capa de Acceso a Datos (Repository):** Crear clases anotadas con `@Repository` que gestionen colecciones (`List` o `Map`) para simular la base de datos de Venues y Events.
2. **Capa de Negocio (Service):** Implementar clases con `@Service` que contengan la lógica de validación (ej. un evento no puede tener nombre vacío) y orquesten la comunicación con el repositorio.
3. **Capa de Presentación (Controller):** Implementar `@RestController` para exponer endpoints de la API.

4. Modelos (POJOs): Definir las entidades básicas con sus atributos:

- `Event`: id, nombre, fecha, descripción.
- `Venue`: id, nombre, dirección, capacidad.

TASK 3

Documentación y Calidad (Testing por Clase)

1. **Swagger/OpenAPI:** Configurar la dependencia `springdoc-openapi-starter-webmvc-ui`. Personalizar los nombres de los endpoints en la interfaz de Swagger.
2. **Unit Testing:** Crear clases de prueba con JUnit 5 y Mockito para testear los Servicios de forma aislada (sin levantar el contexto de Spring).
3. **Manejo de Estados HTTP:** Asegurar que el controlador responda con `201 Created` al registrar y `200 OK` al consultar.

Criterios de aceptación

Escenario 1: Registro Exitoso (Camino Feliz)

- Dado un administrador con datos válidos de un evento.
- Cuando envía una petición `POST /api/events`.
- Entonces el sistema debe:
 1. Validar la información en la capa `@Service`.
 2. Almacenar el objeto en la colección del `@Repository`.
 3. Retornar el objeto creado con un estado 201 Created.

Escenario 2: Intento de Registro Inválido (Camino de Error)

- Dado un evento con el nombre vacío.
- Cuando se intenta registrar.
- Entonces la capa de negocio (`@Service`) debe detectar la falla.
- Y el sistema debe retornar un error controlado (o lanzar excepción) impidiendo que el dato corrupto llegue al repositorio.

Escenario 3: Consulta de Catálogo Vacío (Caso de Borde)

- Dado que no se han cargado datos y el seeder está desactivado.
- Cuando el usuario consulta el listado.
- Entonces el sistema debe retornar una lista vacía `[]` con estado 200 OK, demostrando que la capa de persistencia en memoria está operativa.

Escenario 4: Verificación de Documentación

- Dado que la aplicación está corriendo.
- Cuando el desarrollador accede a `/swagger-ui.html`.
- Entonces debe visualizar todos los métodos `GET` y `POST` documentados y listos para probar.